

Learn Low-Level Design in 25 Days: Day 19— Designing a Food Delivery App (Swiggy/UberEats Style)

You open Swiggy, tap your favorite restaurant, add biryani to the cart, and 30 minutes later, it’s at your door.

TechPreneur

Published October 31, 2025

Free: No

Read on Freedium · original

Contents

Core Concepts — Thinking Like a System Designer 0

The Core Flow 0

Step 1: Identify Core Entities 0

Step 2: Model the Order Lifecycle 0

Step 3: Designing the Restaurant System 0

Step 4: Delivery Partner Matching 0

Step 5: Tracking Delivery in Real-Time 0

Step 6: Payments and Settlement 0

Visual Overview 0

Try It Yourself 0

Summary 0

What's Next 0

Simple? Not quite.

Behind that "Place Order" button lies a symphony of systems — real-time order tracking, restaurant availability, delivery partner matching, payments, and notifications — all working seamlessly under heavy load.

Welcome to **Day 19 of Learn Low-Level Design in 25 Days**, where we'll design one of the most complex and fascinating systems in everyday life — a **Food Delivery App**, inspired by platforms like **Swiggy, Zomato, or UberEats**.

By the end of this session, you'll know how to model such a system at the **class and interaction level** — focusing on **clean abstractions**, **responsibility segregation**, and **real-world object modeling**.



Core Concepts — Thinking Like a System Designer

At its heart, a food delivery platform connects three entities:

1. **Customer** — places and tracks orders.
2. **Restaurant** — accepts and prepares orders.

3. **Delivery Partner** — picks up and delivers orders.

Each of these entities interacts with the system via services and data models that reflect real-world relationships.

The Core Flow

Let's break down a simple order journey:

```
Customer → places order → matched with restaurant →
restaurant confirms → driver assigned →
driver picks up → order delivered → payment settled
```

That's a lot of moving parts. To keep the system manageable, we'll design it using **separation of concerns** — each subsystem focuses on one responsibility.

Step 1: Identify Core Entities

Think about the "nouns" in the problem. These become your classes.

```
User (abstract)
  - Customer
  - DeliveryPartner
  - RestaurantOwner

Restaurant
  - Menu
  - Item

Order
  - OrderStatus
  - Payment

Delivery
  - DeliveryStatus
  - Location
```

Step 2: Model the Order Lifecycle

Let's focus on the **Order** object — the central piece of this puzzle.

```
class Order {
    private String orderId;
    private Customer customer;
    private Restaurant restaurant;
    private List<Item> items;
    private OrderStatus status;
    private Payment payment;
    private Delivery delivery;

    public void placeOrder() { ... }
    public void cancelOrder() { ... }
    public void updateStatus(OrderStatus newStatus) { ... }
}
```

This `Order` is the backbone — everything else (restaurant availability, driver matching, payments) depends on its state transitions.

Step 3: Designing the Restaurant System

Restaurants need to manage menus, handle availability, and confirm orders.

```
class Restaurant {
    private String id;
    private String name;
    private List<Item> menu;
    private boolean isOpen;

    public boolean acceptOrder(Order order) {
        if (!isOpen) return false;
        // Business logic to confirm order
        return true;
    }
}
```

The **RestaurantService** could handle menu management and order confirmations. This separation makes the restaurant independent of the order-processing logic.

Step 4: Delivery Partner Matching

Here's where it gets interesting. When a restaurant accepts an order, the system must **find the nearest delivery partner**.

We can abstract that as a **MatchingService**, using a **Strategy pattern** to switch between matching algorithms (e.g., nearest driver, least busy driver, zone-based matching).

```
interface MatchingStrategy {
    DeliveryPartner findPartner(Order order);
}

class NearestPartnerStrategy implements MatchingStrategy {
    public DeliveryPartner findPartner(Order order) {
        // Fetch drivers near restaurant and pick closest
    }
}

class MatchingService {
    private MatchingStrategy strategy;

    public DeliveryPartner assignPartner(Order order) {
        return strategy.findPartner(order);
    }
}
```

This approach gives flexibility — the app can evolve its matching algorithm without breaking core logic.

Step 5: Tracking Delivery in Real-Time

Once assigned, the **Delivery** object maintains the current location and delivery status.

```
class Delivery {
    private DeliveryPartner partner;
    private DeliveryStatus status;
    private Location currentLocation;

    public void updateLocation(Location loc) { this.currentLocation = loc; }
    public void markDelivered() { this.status = DeliveryStatus.DELIVERED; }
}
```

You can use the **Observer pattern** here — the **Customer** subscribes to delivery updates and gets notified of status changes in real-time.

```
Delivery ---> Customer (Observer)
          ---> Restaurant (Observer)
```

This pattern is what allows your Swiggy app to instantly show "Your order is picked up" without polling.

Step 6: Payments and Settlement

Finally, the **PaymentService** handles transactions.

```
class Payment {
    private String transactionId;
    private double amount;
    private PaymentStatus status;

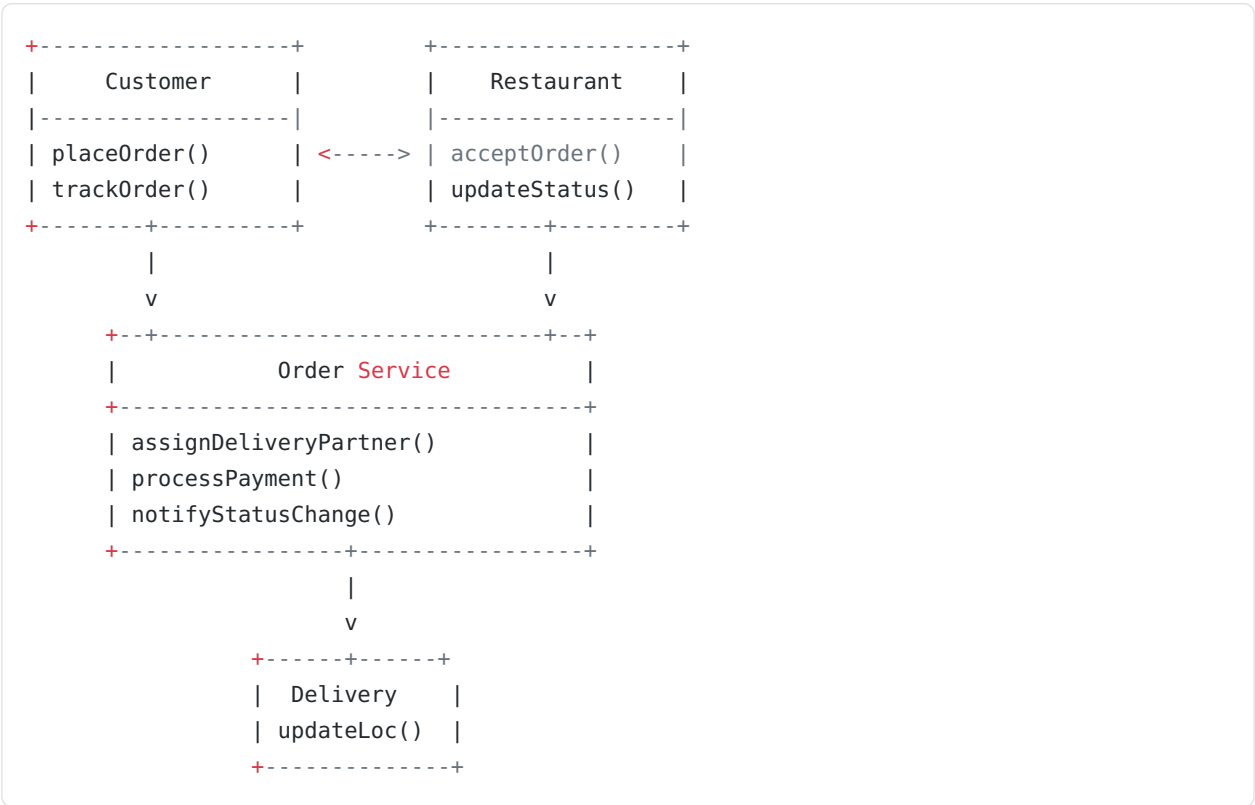
    public boolean processPayment() { ... }
}
```

To avoid coupling, use **interfaces** so that you can switch between payment gateways easily:

```
interface PaymentProcessor {
    boolean pay(double amount);
}

class RazorpayProcessor implements PaymentProcessor { ... }
class StripeProcessor implements PaymentProcessor { ... }
```

Visual Overview



Try It Yourself

1. **Challenge 1:** Extend the design to handle **scheduled orders** (e.g., "deliver at 8:00 PM"). Which new class or field would you introduce?

2. **Challenge 2:** Add a **rating system** for restaurants and delivery partners. How would you ensure ratings don't tightly couple to the `Order` class?

Summary

Let's recap the key lessons:

- Break down complex systems by **responsibility** (Order, Restaurant, Delivery, Payment).
- Use **Strategy** for flexible algorithms (like driver matching).
- Apply **Observer** for real-time updates and loose coupling.
- Always design for **extensibility** — new payment gateways, delivery strategies, or tracking methods should fit in naturally.

When designing at a low level, think about how each class plays its role in the bigger story — just like actors in a film, not all need to know the full script.

What's Next

Tomorrow, we'll shift gears into **Day 20 — API Design Principles in LLD** You'll learn how to expose clean, versioned APIs that bridge frontend and backend systems effectively.

If you're following the full **Learn Low-Level Design in 25 Days** series, here's the list to bookmark and save: https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8 (https://freedium-mirror.cfd/https://techpreneurr.medium.com/list/lowlevel-design-in-25-days-a-practical-bootcamp-for-devs-8829ef3508d8)